

КАФЕДРА «Информационная безопасность» (ИУ8)

по дисциплине «Введение в искусственный интеллект»

Вариант 2

Студент	<u>ИУ8-13М</u>	<u>Бочкарев Михаил Александрович</u>	
	(Группа)	(И. О. Фамилия)	(Подпись, дата)
Преподаватель		<u>Каганов Юрий Тихонович</u>	
		(И. О. Фамилия)	(Подпись, дата)

Оценка:

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Цель работы	3
2 Постановка задачи	4
3 Теоретические сведения	5
3.1 Модель персептрона	5
3.2 Правило обучения	5
3.3 Представление изображения	6
4 Формирование обучающей выборки	7
5 Программная реализация	8
5.1 Структура программы	8
5.2 Обучение и распознавание	8
5.3 Центрирование введённого изображения	9
5.4 Отказ от распознавания некорректного изображения	9
5.5 Графический интерфейс	10
6 Результаты тестирования	11
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13
ПРИЛОЖЕНИЕ А Исходный код программы	14

ВВЕДЕНИЕ

В работе рассматривается простейшая задача распознавания печатных цифр от 0 до 9 с помощью однослойного персептрона. Каждая цифра задаётся бинарным растровым изображением размером 3×5 . Закрашенная клетка кодируется единицей, незакрашенная — нулём. Таким образом, один образ описывается вектором из 15 бинарных признаков.

Для многоклассового распознавания используется десять выходных нейронов, каждый из которых соответствует одной цифре. В процессе обучения веса корректируются так, чтобы выход правильного класса становился больше выходов остальных классов. После обучения реализуется автоматическая проверка на эталонных образах и графический интерфейс, позволяющий сформировать цифру мышью в поле 3×5 и передать её на распознавание.

Дополнительно в программе предусмотрена простая защита от заведомо некорректного изображения: результат отвергается, если максимальный выход персептрона недостаточно велик, его отрыв от второго класса слишком мал либо образ слишком сильно отличается от ближайшего эталона.

1 Цель работы

Цель работы — изучить принцип работы и обучения персептрона на примере задачи классификации бинарных изображений печатных цифр.

Для достижения поставленной цели необходимо:

- а) сформировать обучающие изображения цифр 0–9 размером 3×5 ;
- б) реализовать многоклассовый персептрон;
- в) выполнить обучение модели на эталонных изображениях;
- г) проверить корректность распознавания обучающих данных;
- д) разработать простой графический интерфейс для ручного ввода изображения;
- е) предусмотреть отказ от распознавания при подаче изображения, не похожего на цифру.

2 Постановка задачи

Необходимо реализовать на языке Python программу распознавания цифр от 0 до 9. Вход программы представляет собой бинарное изображение размером 3×5 . Каждая клетка изображения принимает одно из двух значений:

$$x_i \in \{0, 1\}, \quad i = 1, \dots, 15, \quad (1)$$

где 0 соответствует пустой клетке, а 1 — закрашенной.

Для каждого класса $k \in \{0, 1, \dots, 9\}$ должен быть сформирован выход персептрона. Распознанной считается цифра, выход которой имеет наибольшее значение. Обучение выполняется на десяти эталонных изображениях. После завершения обучения требуется проверить распознавание всех эталонов и предоставить возможность ручного тестирования через графический интерфейс.

Вариант работы — 2.

3 Теоретические сведения

3.1 Модель персептрона

Персептрон является одной из простейших моделей искусственного нейрона. В классическом случае он вычисляет взвешенную сумму входных признаков и принимает решение на основе полученного значения [1].

В данной работе один объект описывается 15 входами. Для распознавания десяти классов используется десять наборов весов. Для класса k вычисляется оценка

$$s_k = b_k + \sum_{i=1}^{15} w_{ki}x_i, \quad (2)$$

где w_{ki} — вес i -го пикселя для класса k , b_k — смещение соответствующего выходного нейрона.

Решение определяется как индекс максимальной оценки:

$$\hat{y} = \arg \max_{k \in \{0, \dots, 9\}} s_k. \quad (3)$$

Таким образом, архитектура модели содержит 15 входов и 10 выходов. Отдельного скрытого слоя нет.

3.2 Правило обучения

Перед началом обучения все веса и смещения инициализируются нулевыми значениями. Для очередного эталона вычисляются оценки всех десяти классов. Затем определяется наиболее сильный неверный класс. Если оценка правильного класса не превосходит оценку неверного класса с заданным запасом, веса корректируются.

Для правильного класса y используется правило

$$\mathbf{w}_y \leftarrow \mathbf{w}_y + \eta \mathbf{x}, \quad (4)$$

$$b_y \leftarrow b_y + \eta, \quad (5)$$

а для наиболее сильного ошибочного класса j

$$\mathbf{w}_j \leftarrow \mathbf{w}_j - \eta \mathbf{x}, \quad (6)$$

$$b_j \leftarrow b_j - \eta. \quad (7)$$

В программе используется норма обучения $\eta = 0,1$. Дополнительно введён запас между правильным и ошибочным выходом $m = 0,5$. Коррекция выполняется, если

$$s_y \leq s_j + m. \quad (8)$$

Такой вариант правила обучения заставляет правильный класс не только стать максимальным, но и получить некоторый отрыв от ближайшего конкурента.

3.3 Представление изображения

Растр размером 3×5 преобразуется в одномерный вектор длины 15. Например, изображение цифры 2

```

111
001
111
100
111

```

соответствует вектору

$$\mathbf{x} = (1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1)^T. \quad (9)$$

4 Формирование обучающей выборки

В качестве обучающей выборки используются десять эталонных печатных цифр. Размер 3×5 выбран как минимальный удобный симметричный растр, позволяющий визуально различить все десять цифр.

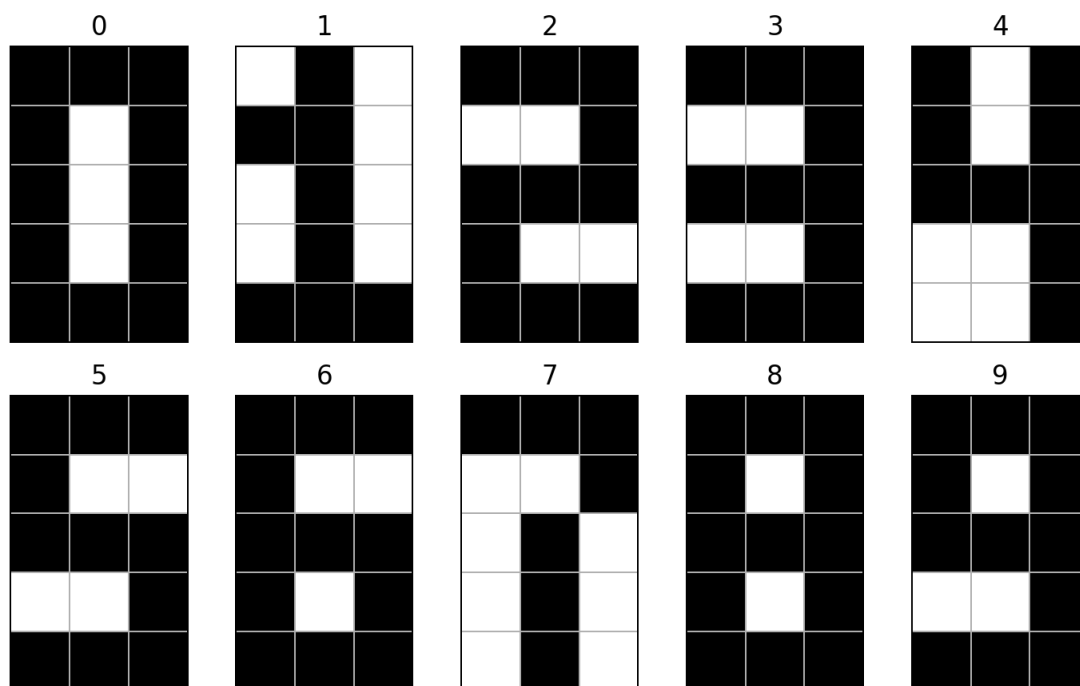


Рисунок 1 – Эталонные изображения цифр размером 3×5

Каждый эталон используется как один обучающий объект. Меткой объекта является номер изображённой цифры. Обучение и контроль на данном этапе выполняются на корректных печатных образах.

5 Программная реализация

5.1 Структура программы

Программа реализована на языке Python. Для математических операций сторонние библиотеки не используются: вычисление взвешенных сумм и изменение весов выполнены стандартными средствами языка. Графическая часть построена с использованием модуля `tkinter`, входящего в стандартную библиотеку Python [2].

Основные этапы работы программы:

- а) задание десяти эталонных цифр;
- б) преобразование строкового представления в бинарные векторы;
- в) создание матрицы весов размером 10×15 и десяти смещений;
- г) обучение персептрона до прекращения корректировок;
- д) автоматическая проверка всех эталонных изображений;
- е) запуск графического интерфейса для ручного тестирования.

5.2 Обучение и распознавание

Функция `scores` вычисляет десять выходных оценок. Функция `predict` возвращает номер класса с максимальным значением. Обучение выполняется последовательно по всем десяти образам. Если правильный класс не имеет необходимого преимущества, веса правильного и ошибочного классов изменяются в противоположных направлениях.

Максимальное число эпох ограничено значением 500. При отсутствии корректировок обучение завершается досрочно. Для выбранных эталонов условие остановки было достигнуто на 41-й эпохе.

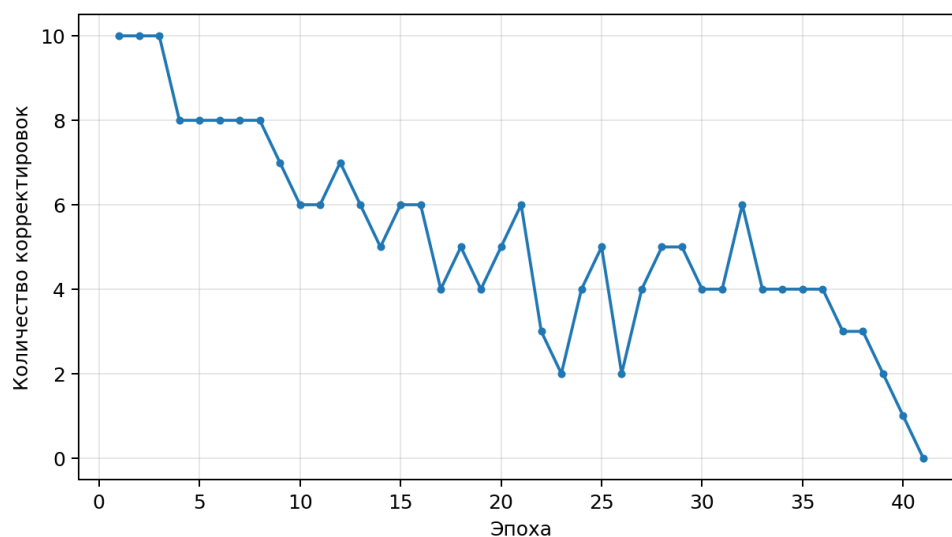


Рисунок 2 – Изменение числа корректировок весов по эпохам

5.3 Центрирование введённого изображения

При ручном вводе пользователь может сформировать символ со смещением относительно центра сетки. Перед передачей изображения на вход персептрона выполняется простая нормализация положения. Определяется минимальный прямоугольник, содержащий все закрашенные клетки, после чего этот прямоугольник переносится в центральную область сетки.

Такая операция не изменяет форму символа, но уменьшает зависимость результата от его положения в поле. Следует отметить, что инвариантность к сдвигу в данном случае достигается не самим персептроном, а предварительной обработкой входного изображения.

5.4 Отказ от распознавания некорректного изображения

Обычный многоклассовый персептрон всегда выбирает один из десяти классов, даже если на вход подан произвольный набор клеток. Для устранения этого недостатка после вычисления выходов введены три дополнительные проверки:

- а) максимальная оценка должна быть не меньше заданного порога;
- б) разность между первым и вторым по величине выходами должна быть достаточной;
- в) расстояние Хэмминга до ближайшего эталона не должно превышать допустимое значение.

В программе используются параметры

$$s_{\min} = 0,9, \quad \Delta_{\min} = 0,35, \quad d_{\max} = 3. \quad (10)$$

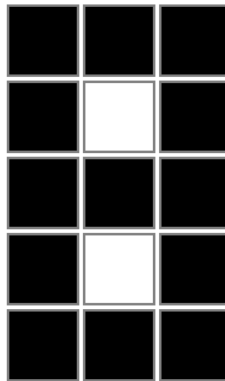
Если хотя бы одно условие нарушается, интерфейс выводит сообщение «не цифра». Эта проверка является вспомогательной: номер распознанной цифры по-прежнему определяется выходами обученного персептрона.

5.5 Графический интерфейс

Интерфейс содержит поле из 15 кнопок, расположенных в виде матрицы 3×5 . Нажатие на клетку переключает её состояние между пустым и закрашенным. Кнопка «Распознать» формирует входной вектор и передаёт его модели, кнопка «Очистить» сбрасывает поле.

Персептрон: цифры 3×5

Нарисуйте цифру 0–9



Распознать

Очистить

Результат: 8

Рисунок 3 – Схема графического интерфейса программы

В нижней части окна отображается результат и диагностическая информация: максимальная оценка, отрыв от второго класса, расстояние до ближайшего эталона и значения выходов всех десяти нейронов.

6 Результаты тестирования

После обучения выполнена автоматическая проверка на всех десяти эталонных изображениях. Полученные результаты приведены в таблице 1.

Таблица 1 – Результаты распознавания эталонных цифр

Цифра	Результат	s_{max}	s_2	Δ
0	0	1,8	0,4	1,4
1	1	2,7	1,5	1,2
2	2	1,7	1,0	0,7
3	3	2,2	0,5	1,7
4	4	2,8	0,6	2,2
5	5	2,5	0,9	1,6
6	6	2,2	0,8	1,4
7	7	2,1	0,9	1,2
8	8	1,0	0,3	0,7
9	9	1,2	0,6	0,6

Все десять эталонов были классифицированы корректно. Точность на проверочной последовательности составила

$$A = \frac{10}{10} \cdot 100\% = 100\%. \quad (11)$$

В ходе обучения на 41-й эпохе число корректировок стало равно нулю. Это означает, что для всех обучающих примеров правильный класс получил требуемое преимущество над наиболее сильным ошибочным классом.

Ручная проверка через интерфейс позволяет дополнительно убедиться в работоспособности всей цепочки: ввод изображения, центрирование, вычисление выходов, выбор класса и вывод результата выполняются в одной программе.

ЗАКЛЮЧЕНИЕ

В ходе выполнения задания реализована программа распознавания печатных цифр с использованием однослойного многоклассового персептрона. Каждое изображение представляется 15 бинарными признаками, соответствующими клеткам раstra 3×5 . Для классификации используются десять выходных нейронов, по одному на каждый класс.

Модель была обучена на десяти эталонных изображениях. Обучение завершилось на 41-й эпохе, после чего все десять эталонных цифр распознавались корректно, что соответствует точности 100% на используемой проверочной последовательности.

Для тестирования разработан графический интерфейс с интерактивной сеткой 3×5 . Перед распознаванием выполняется центрирование изображения. Также реализована дополнительная проверка, позволяющая отказаться от выбора класса при подаче изображения, существенно отличающегося от известных цифр.

Полученная реализация демонстрирует основные этапы решения задачи классификации с помощью персептрона: формирование признакового вектора, вычисление линейных выходов, корректировку весов по ошибке и выбор класса по максимальному выходу.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rosenblatt F. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain // Psychological Review. — 1958. — Т. 65, № 6. — С. 386—408. — DOI: 10.1037/h0042519.
2. Python Software Foundation. tkinter — Python interface to Tcl/Tk. — URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения 06.10.2026).

ПРИЛОЖЕНИЕ А

Исходный код программы

Ниже приведён исходный код разработанной программы.

```
1 import tkinter as tk
2
3 ROWS = 5
4 COLS = 3
5 INPUTS = ROWS * COLS
6 CLASSES = 10
7
8 LEARNING_RATE = 0.1
9 MAX_EPOCHS = 500
10 TRAIN_MARGIN = 0.5
11
12 SCORE_THRESHOLD = 0.9
13 GAP_THRESHOLD = 0.35
14 MAX_TEMPLATE_DISTANCE = 3
15
16
17 DIGITS = {
18     0: [
19         "111",
20         "101",
21         "101",
22         "101",
23         "111",
24     ],
25     1: [
26         "010",
27         "110",
28         "010",
29         "010",
30         "111",
31     ],
32     2: [
33         "111",
34         "001",
35         "111",
36         "100",
37         "111",
38     ],
39     3: [
40         "111",
41         "001",
42         "111",
43         "001",
44         "111",
45     ],
46     4: [
```

```

47         "101",
48         "101",
49         "111",
50         "001",
51         "001",
52     ],
53     5: [
54         "111",
55         "100",
56         "111",
57         "001",
58         "111",
59     ],
60     6: [
61         "111",
62         "100",
63         "111",
64         "101",
65         "111",
66     ],
67     7: [
68         "111",
69         "001",
70         "010",
71         "010",
72         "010",
73     ],
74     8: [
75         "111",
76         "101",
77         "111",
78         "101",
79         "111",
80     ],
81     9: [
82         "111",
83         "101",
84         "111",
85         "001",
86         "111",
87     ],
88 }
89
90
91 def image_to_vector(image):
92     return [int(pixel) for row in image for pixel in row]
93
94
95 def center_image(vector):
96     matrix = [
97         vector[row * COLS:(row + 1) * COLS]
98         for row in range(ROWS)
99     ]

```

```

100
101     points = [
102         (row, col)
103         for row in range(ROWS)
104         for col in range(COLS)
105         if matrix[row][col] == 1
106     ]
107
108     if not points:
109         return vector[:]
110
111     min_row = min(row for row, _ in points)
112     max_row = max(row for row, _ in points)
113     min_col = min(col for _, col in points)
114     max_col = max(col for _, col in points)
115
116     height = max_row - min_row + 1
117     width = max_col - min_col + 1
118
119     target_row = (ROWS - height) // 2
120     target_col = (COLS - width) // 2
121
122     result = [[0] * COLS for _ in range(ROWS)]
123
124     for row, col in points:
125         new_row = row - min_row + target_row
126         new_col = col - min_col + target_col
127
128         if 0 <= new_row < ROWS and 0 <= new_col < COLS:
129             result[new_row][new_col] = 1
130
131     return [pixel for row in result for pixel in row]
132
133
134 def hamming_distance(a, b):
135     return sum(x != y for x, y in zip(a, b))
136
137
138 X = [image_to_vector(DIGITS[digit]) for digit in range(CLASSES)]
139 Y = list(range(CLASSES))
140
141
142 class Perceptron:
143     def __init__(self):
144         self.weights = [[0.0] * INPUTS for _ in range(CLASSES)]
145         self.bias = [0.0] * CLASSES
146
147     def scores(self, x):
148         result = []
149
150         for digit in range(CLASSES):
151             value = self.bias[digit]

```

```

153         for i in range(INPUTS):
154             value += self.weights[digit][i] * x[i]
155
156         result.append(value)
157
158     return result
159
160     def predict(self, x):
161         values = self.scores(x)
162         return max(range(CLASSES), key=lambda digit: values[digit])
163
164     def train(self, X, Y):
165         for epoch in range(MAX_EPOCHS):
166             updates = 0
167
168             for x, answer in zip(X, Y):
169                 values = self.scores(x)
170
171                 wrong = max(
172                     (digit for digit in range(CLASSES) if digit != answer),
173                     key=lambda digit: values[digit]
174                 )
175
176                 if values[answer] <= values[wrong] + TRAIN_MARGIN:
177                     for i in range(INPUTS):
178                         self.weights[answer][i] += LEARNING_RATE * x[i]
179                         self.weights[wrong][i] -= LEARNING_RATE * x[i]
180
181                     self.bias[answer] += LEARNING_RATE
182                     self.bias[wrong] -= LEARNING_RATE
183                     updates += 1
184
185             print(f"Эпоха {epoch + 1}: корректировок {updates}")
186
187             if updates == 0:
188                 print("Обучение завершено.\n")
189                 return
190
191         print("Достигнуто максимальное число эпох.\n")
192
193
194 model = Perceptron()
195 model.train(X, Y)
196
197
198 print("Проверка на обучающих данных:")
199
200 correct = 0
201
202 for x, answer in zip(X, Y):
203     prediction = model.predict(x)
204     print(f"Цифра {answer} -> распознано {prediction}")
205

```

```

206         if prediction == answer:
207             correct += 1
208
209     print(f"Точность: {correct}/{CLASSES}\n")
210
211
212     class App:
213         def __init__(self, root):
214             self.root = root
215             self.root.title("Персептрон: цифры 3x5")
216             self.root.resizable(False, False)
217
218             self.cells = [0] * INPUTS
219             self.buttons = []
220
221             title = tk.Label(
222                 root,
223                 text="Нарисуйте цифру 0-9",
224                 font=("Arial", 16, "bold")
225             )
226             title.pack(pady=(15, 10))
227
228             grid = tk.Frame(root)
229             grid.pack(padx=20, pady=5)
230
231             for row in range(ROWS):
232                 for col in range(COLS):
233                     index = row * COLS + col
234
235                     button = tk.Button(
236                         grid,
237                         width=5,
238                         height=2,
239                         bg="white",
240                         activebackground="gray80",
241                         command=lambda i=index: self.toggle_cell(i)
242                     )
243
244                     button.grid(row=row, column=col, padx=2, pady=2)
245                     self.buttons.append(button)
246
247             actions = tk.Frame(root)
248             actions.pack(pady=12)
249
250             tk.Button(
251                 actions,
252                 text="Распознать",
253                 width=14,
254                 command=self.recognize
255             ).grid(row=0, column=0, padx=5)
256
257             tk.Button(
258                 actions,

```

```

259         text="Очистить",
260         width=14,
261         command=self.clear
262     ).grid(row=0, column=1, padx=5)
263
264     self.result = tk.Label(
265         root,
266         text="Результат: -",
267         font=("Arial", 18, "bold")
268     )
269     self.result.pack(pady=(5, 5))
270
271     self.info = tk.Label(
272         root,
273         text="",
274         font=("Arial", 10),
275         justify="left"
276     )
277     self.info.pack(padx=15, pady=(0, 15))
278
279     def toggle_cell(self, index):
280         self.cells[index] = 1 - self.cells[index]
281
282         if self.cells[index]:
283             self.buttons[index].configure(bg="black")
284         else:
285             self.buttons[index].configure(bg="white")
286
287     def clear(self):
288         self.cells = [0] * INPUTS
289
290         for button in self.buttons:
291             button.configure(bg="white")
292
293         self.result.configure(text="Результат: -")
294         self.info.configure(text="")
295
296     def recognize(self):
297         x = center_image(self.cells)
298
299         if sum(x) == 0:
300             self.result.configure(text="Результат: не цифра")
301             self.info.configure(text="Поле пустое.")
302             return
303
304         values = model.scores(x)
305         order = sorted(range(CLASSES), key=lambda digit: values[digit], reverse=True)
306
307         best_digit = order[0]
308         best_score = values[best_digit]
309         second_score = values[order[1]]
310         gap = best_score - second_score
311

```

```

312         distance = min(hamming_distance(x, template) for template in X)
313
314     is_digit = (
315         best_score >= SCORE_THRESHOLD
316         and gap >= GAP_THRESHOLD
317         and distance <= MAX_TEMPLATE_DISTANCE
318     )
319
320     if is_digit:
321         self.result.configure(text=f"Результат: {best_digit}")
322     else:
323         self.result.configure(text="Результат: не цифра")
324
325     scores_text = " ".join(
326         f"{digit}: {values[digit]:.1f}"
327         for digit in range(CLASSES)
328     )
329
330     self.info.configure(
331         text=(
332             f"Максимальный выход: {best_score:.2f}\n"
333             f"Отрыв от второго: {gap:.2f}\n"
334             f"Отклонение от ближайшего эталона: {distance} клеток\n\n"
335             f"Выходы персептрона:\n{scores_text}"
336         )
337     )
338
339
340 root = tk.Tk()
341 App(root)
342 root.mainloop()

```